

Abstract

Object-oriented programming remains one of the most widely adopted paradigms for developing maintainable and scalable software systems. This project focuses on the design and implementation of a Smart Library Management System (SmartLib) using modern Java 17 features and established software engineering principles. The project applies SOLID principles to create a modular architecture while incorporating Gang-of-Four design patterns, including Builder, Decorator, and Observer. Advanced Java concepts such as generics, bounded wildcards, the Stream API, records, sealed classes, pattern matching, and concurrency utilities are also explored. Thread-safe processing was implemented to support concurrent book return operations while ensuring consistency and reliability. Functional programming techniques were used to simplify data processing and improve readability. The final solution demonstrates how modern Java supports robust object-oriented design through abstraction, reusability, extensibility, and maintainability. The project highlights the practical importance of combining design principles, software patterns, and modern language features in real-world applications.

Introduction and Design Overview

The SmartLib system was developed as a comprehensive library management solution designed to handle books, members, loans, reservations, notifications, fines, and event processing. The architecture was intentionally structured around SOLID principles to encourage low coupling and high cohesion.

The design separates responsibilities across domain entities, repositories, services, policies, and infrastructure components. Abstractions such as `Repository<T, ID>`, `BorrowingPolicy`, `LoanService`, and `Notifiable` allow the system to remain flexible and easily extensible. The use of dependency injection ensures that high-level business logic remains independent of implementation details. This design approach supports future enhancements without requiring major modifications to existing code.

Question 1.1 – SOLID Analysis and Refactoring

The provided `LibraryManager` class violates multiple SOLID principles.

Single Responsibility Principle (SRP)

Violation

The class performs multiple unrelated responsibilities:

- Loan management
- Fine calculation
- Email notification

- File persistence
- Report generation

Examples:

```
public void returnBook(String loanId)
```

and

```
public void generateReport(String format)
```

Each of these responsibilities changes for different reasons.

Why It Is Problematic

Any modification to reporting, email delivery, file storage, or fine calculation forces changes within the same class. This increases maintenance complexity and testing difficulty.

Refactoring

Separate classes should be introduced:

- LoanService
- FineCalculator
- NotificationService
- LoanRepository
- ReportGenerator

Open/Closed Principle (OCP)

Violation

```
if(format.equals("PDF"))  
...  
else if(format.equals("CSV"))  
...
```

Adding a new report type requires modifying existing code.

Why It Is Problematic

Every new format introduces additional conditional branches, increasing complexity and violating extensibility.

Refactoring

Introduce:

interface ReportGenerator

with implementations:

PdfReportGenerator

CsvReportGenerator

ConsoleReportGenerator

New formats can then be added without changing existing logic.

Dependency Inversion Principle (DIP)

Violation

The class directly creates infrastructure objects:

new FileWriter(...)

new Properties()

Why It Is Problematic

Business logic becomes tightly coupled to low-level implementation details.

Refactoring

Depend on abstractions:

NotificationService

LoanRepository

These dependencies are injected through constructors.

Interface Segregation Principle (ISP)

Although not directly visible, the monolithic design forces clients to depend on reporting, persistence, and notification functionality even when they only require loan processing.

Smaller focused interfaces resolve this issue.

Liskov Substitution Principle (LSP)

The code does not explicitly violate LSP, but the lack of abstractions makes future substitutability difficult.

By introducing interfaces, any implementation can be substituted without changing program behaviour.

Refactored Architecture

The improved design consists of:

LoanService

└─ FineCalculator

└─ NotificationService

└─ LoanRepository

└─ ReportGenerator

Each component has a single responsibility and can be independently tested using dependency injection and mocking frameworks such as Mockito.

Question 2.1 – Builder Pattern

The Builder pattern is appropriate for constructing Loan objects because several optional attributes exist alongside mandatory ones.

Why Builder Is Better Than Telescoping Constructors

A telescoping constructor would require many overloaded constructors:

Loan(member, book)

Loan(member, book, notes)

Loan(member, book, notes, renewalCount)

Loan(member, book, notes, renewalCount, referenceCode)

This quickly becomes difficult to read and maintain.

The Builder pattern provides:

- Readability
- Immutability

- Validation before object creation
- Flexible optional parameters

Example:

```
Loan loan = new LoanBuilder(member, book)
    .notes("Renewed copy")
    .renewalCount(2)
    .referenceCode("LIB-2026")
    .build();
```

Another Suitable Builder Candidate

The Notification entity would also benefit from the Builder pattern because:

- Channel is optional.
- Delivery metadata may vary.
- Future notification types can be added without modifying constructors.

Question 2.2 – Decorator Pattern

The Decorator pattern allows additional behaviour to be attached dynamically to a NotificationService without modifying the original implementation.

Benefits include:

- Runtime flexibility
- Reusability
- Separation of concerns

Example composition:

```
new RateLimitingDecorator(
    new RetryDecorator(
        new LoggingDecorator(
            new EmailNotificationService()
        ),
        3
    ),
```

5
);

Decorator vs Inheritance

Decorator

Advantages:

- Runtime composition
- Reduced class explosion
- Better adherence to OCP

Inheritance

Advantages:

- Simpler when behaviour is fixed
- Lower runtime overhead

Inheritance is preferable when behaviour is stable and unlikely to vary.

Question 2.3 – Observer Pattern

The Observer pattern supports event-driven communication between SmartLib components.

Example events:

- BOOK_RETURNED
- FINE_IMPOSED
- RESERVATION_EXPIRED

Observers:

- OverdueFineListener
- MemberNotificationListener
- AuditLogListener

Benefits:

- Loose coupling
 - Extensibility
 - Independent event handling
-

Synchronous vs Asynchronous Event Bus

Synchronous

Advantages:

- Simpler implementation
- Easier debugging
- Predictable execution order

Disadvantages:

- Slow listeners block others

Asynchronous

Advantages:

- Better scalability
- Improved responsiveness
- Higher throughput

Disadvantages:

- Increased complexity
- Harder debugging

For SmartLib, synchronous processing is sufficient for small deployments, while larger systems would benefit from asynchronous event processing.

Question 3.2 – Generics and Type Erasure

What Is Type Erasure?

Java generics exist only during compilation.

At runtime:

Catalogue<Book>

becomes:

Catalogue

This process is known as type erasure.

Why the Example Compiles

```
Catalogue<Book> bookCat = new Catalogue<>();  
Catalogue raw = bookCat;  
raw.add("not a book");
```

The compiler issues a warning because raw types disable generic type checking.

At runtime, heap pollution occurs because a String enters a collection intended for Book objects.

This may eventually result in a ClassCastException.

PECS Principle

Producer Extends Consumer Super:

Producer

? extends T

Used when data is being read.

Consumer

? super T

Used when data is being written.

This improves type safety while maintaining flexibility.

Stream API and Functional Programming Reflections

The Stream API significantly simplifies data processing operations.

Examples include:

- Grouping loans by membership type
- Counting book borrow frequency
- Partitioning books by availability
- Creating author statistics

Benefits observed:

1. Reduced boilerplate code.
2. Improved readability.
3. Easier parallelization.
4. Better declarative programming style.

However, overly complex stream pipelines may reduce readability if excessive chaining is used.

Question 5.3 – Concurrency Design Decisions

Race Condition

A race condition occurs when multiple threads modify shared data without synchronization.

Example:

Two members attempt to borrow the final available copy simultaneously.

Without locking:

`availableCopies--;`

may execute incorrectly.

Deadlock

A deadlock occurs when two threads wait indefinitely for resources held by each other.

Example:

Thread A locks Inventory.

Thread B locks LoanService.

Each waits for the other's lock.

Livelock

A livelock occurs when threads remain active but continually interfere with one another without making progress.

Example:

Two borrowing threads repeatedly release and reacquire locks to avoid conflict.

Why ReentrantReadWriteLock Improves Performance

The inventory system is read-heavy.

Advantages:

- Multiple concurrent readers.
- Exclusive write access only when required.
- Higher throughput than synchronized.

This is particularly valuable in library systems where availability checks occur far more frequently than inventory updates.

ConcurrentHashMap vs synchronizedMap

ConcurrentHashMap is generally preferred because:

- Better scalability.
- Reduced lock contention.
- Segment-based concurrency.
- Improved performance under heavy load.

synchronizedMap locks the entire collection, limiting throughput.

Question 6 – Modern Java Features

Advantages of Sealed Interfaces

Sealed interfaces provide controlled inheritance.

Benefits:

- Compiler-enforced hierarchy.
- Improved readability.
- Safer pattern matching.
- Better modelling of finite state systems.

LoanResult is an ideal candidate because only a limited number of outcomes exist.

Appropriate Uses of Records

Suitable:

1. Book
2. Fine
3. Notification

Reasons:

- Immutable data.
 - Simple state containers.
 - Automatic equals(), hashCode(), and toString().
-

When Mutable Classes Are Better

Suitable:

1. Catalogue
2. InventoryManager
3. LibraryEventBus

Reasons:

- State changes frequently.
- Requires synchronization.
- Maintains dynamic collections.

Java Platform Module System (JPMS)

JPMS improves:

- Encapsulation
- Security
- Dependency management
- Startup performance

Example:

```
module smartlib {  
    exports smartlib.domain;  
    exports smartlib.patterns;  
    exports smartlib.functional;  
  
    requires java.base;  
    requires java.logging;  
}
```

Modules expose only public APIs while hiding internal implementation details.

Conclusion

This project demonstrated how modern Java supports robust object-oriented software development through SOLID principles, design patterns, generics, functional programming, concurrency utilities, and modern language features. The SmartLib system was designed to be modular, maintainable, and scalable while adhering to established software engineering best practices. The Builder, Decorator, and Observer patterns improved extensibility, while generics and streams enhanced type safety and code expressiveness. Concurrency mechanisms ensured thread-safe processing of library operations under concurrent workloads. Modern Java features such as records, sealed interfaces, and pattern matching reduced boilerplate code and improved readability. Overall, the project reinforced the importance of

designing software around clear responsibilities, abstractions, and collaboration between components rather than focusing solely on implementation details.

References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*.
2. Bloch, J. (2018). *Effective Java (3rd Edition)*.
3. Martin, R. C. (2017). *Clean Architecture*.
4. Urma, R., Fusco, M., & Mycroft, A. (2018). *Modern Java in Action*.
5. Naftalin, M., & Wadler, P. (2006). *Java Generics and Collections*.
6. Oracle Java 17 Documentation.

This gives you the report/theory sections. The coding sections (Builder, Decorator, Observer, Generics, Streams, Concurrency, and Modern Java implementations) account for most of the marks and would require the actual Java source files if you want a complete submission package.